R97: Sound Constant Folding for JavaScript Deobfuscation via Target-Runtime Awareness

Anonymous Author(s)
Submission-ready draft

Abstract

JavaScript deobfuscators and minifiers simplify code by evaluating expressions offline and folding them into constants—a transformation that is sound only if the evaluation environment matches the environment where the code will actually run. In reality, the same bundle is deployed to heterogeneous hosts: WeChat mini-programs, browsers, and Node.js servers. Environment-sniffing branches such as `if (typeof wx !== 'undefined')` evaluate to `"undefined"` in an offline Node sandbox, so mainstream deobfuscators delete the very branches that carry core business logic, and the rewritten code crashes when redeployed. We present **R97**, a *target-runtime-aware folding guard* that intercepts constant folding and dead-code elimination in downstream deobfuscators. R97 rests on an orthogonality axiom—what a program *requires* (derived from AST and binding analysis) must never be conflated with where a program *runs* (derivable only from evidence external to the code)—and issues a three-valued verdict per folding point: FOLD (all target hosts observed equivalent), PROTECT (measured conflict), or UNKNOWN (insufficient evidence; conservatively blocked, with the uncertainty source attributed so future evidence can flip it). Verdicts are grounded in a four-host probe matrix (an isolated-VM language baseline, real Node.js, real Edge via CDP, and a real WeChat DevTools AppService) running a single shared oracle implementation, and independently validated by a Level-2 transformation oracle that executes transformed programs and checks observational equivalence—termination state, scalar results, and normalized exception identifiers. On a corpus of 1,572 real WeChat mini-program packages (1.2M program points, 1,248 distinct runtime entities) plus official Node.js and web-platform-tests corpora, R97 approves 30.6–75.2% of folding candidates depending on sampling calibre, while **zero** of its approved folds land on any of the 2,118 measured cross-host differential records; a hard set of 154 known-divergent entities is blocked with 100% recall. Under controlled obfuscation of 112 real files, decisions migrate only in the conservative direction across 4,210 paired entities, again with zero unsound folds. R97 processes 926–3,147 decisions per second with an 8.2 MB steady-state heap for whole-file analysis, and its gate intercepts every behavior-changing rewrite produced by real Terser and webcrack runs. We further contribute a documented catalogue of five extrapolation failures in cross-environment measurement—each caught by an independent detection channel during development—with reusable checks that we argue are a prerequisite for believing any soundness number in cross-host program analysis.

1 Introduction

Modern JavaScript is written once and deployed everywhere. Cross-platform frameworks compile a single source tree into bundles that run as WeChat mini-programs, inside browsers, and on Node.js servers. To analyze, audit, or recover such bundles, security practitioners rely on deobfuscators (webcrack [4], JSIMPLIFIER [16]) and minifiers (Terser [1]) that *simplify* code: they evaluate expressions offline, fold the results into constants, and prune branches that appear unreachable. The transformation inherits a classical assumption from compiler optimization—folding a constant expression is semantics-preserving *if* the evaluator's semantics match the runtime's semantics. For heterogeneous deployments, the assumption is false, and the tools that rely on it are structurally blind to their own error: nothing in a Node sandbox tells the folder that the code it is shrinking will never run in a Node sandbox.

The mismatch has a canonical mechanism. JavaScript's `typeof` operator returns `"undefined"` for undeclared identifiers instead of throwing, which makes `typeof X !== 'undefined'` the universal cross-host dispatch idiom:

Listing 1: Real-world transport-layer selection (Axios/Taro pattern).

```
if (typeof wx !== 'undefined' && wx.request) {
    transport = wechatAdapter; // WeChat mini-program
} else if (typeof XMLHttpRequest !== 'undefined') {
    transport = xhrAdapter; // browser
} else if (typeof process !== 'undefined') {
    transport = nodeHttpAdapter; // Node.js server
}
```

Evaluated offline in a Node sandbox, `typeof wx` is `"undefined"`, so the deobfuscator prunes the WeChat branch as dead code. The rewritten bundle deploys back to WeChat and crashes on its first network call. In the evasion scenario documented by FV8 [11], the consequences invert: malicious code probes `navigator.webdriver` and releases its payload only on real victims; an analysis sandbox that folds environment branches *deletes the malicious branch and keeps the decoy*, producing false negatives in malware analysis. This is not an exotic corner: running one shared probe implementation across four real hosts, we measured **208 divergent entities (785 records)** on 1,572 real WeChat mini-program packages, **184 entities (553 records)** on the official Node.js test corpus, and **273 entities (780 records)** on the W3C web-platform-tests corpus. Divergence is the norm, not the exception.

Existing work does not close this gap. Obfuscation-*testing* frameworks such as OBsmith [7] find semantic-breaking bugs in obfuscators, but their equivalence oracles compare pre- and post-obfuscation programs *within one local Node.js process*—a homogeneous-single-runtime assumption that is precisely what breaks when the analyzed code targets WeChat or the browser. Industrial deobfuscation pipelines such as Google's CASCADE [6] sandbox prelude decryption and push constant propagation hard, but never model where the *output* will be deployed. The defense seen in practice—a blocklist of known-dangerous symbols—fails measurably: our ablations show two representative blacklist designs escape 115 and 239

of 785 measured-differential records respectively, because no fixed list anticipates which entity the *next* bundle will probe.

R97 (Reference Core v1) is a *guard*, not a deobfuscator: it intercepts each folding decision a downstream tool wants to make and returns one of three verdicts—FOLD (safe on every confirmed target host), PROTECT (measured cross-host conflict; the branch must be preserved), or UNKNOWN (insufficient evidence; conservatively blocked, with the uncertainty source machine-attributed so that future evidence can flip it to FOLD). Its design rests on four pillars:

(1) **An orthogonality axiom** (§3–§4.2): the *required* runtime derived from code (`wx` appears ⇒ the code needs WeChat capability) must never be conflated with the *target* runtime (`wx` appears ⇏ the code runs on WeChat). Only evidence external to the analyzed code—packaging manifests, build fingerprints, deployment configurations—can confirm a target, and only `confirmed` targets can authorize folds. The axiom is enforced structurally: circular evidence never reaches the decision engine.

(2) **Minimal semantic contracts** (§4.3): for each folding point, the needed behavioral evidence is projected from the transformation kind and usage context into a minimal set of observable dimensions. The probe matrix publishes a non-destructive triple—existence, type, callability—for every entity; dimensions that would require *invoking* host APIs are deliberately not produced. Non-deterministic language built-ins (`Date.now`, `Math.random`) are trapped by demanding a dimension (`return_value`) that no offline probe can soundly supply, forcing UNKNOWN by construction rather than by an ad-hoc blocklist.

(3) **A four-host probe matrix with one shared oracle implementation** (§4.6): an isolated-VM language baseline, real Node.js, real Edge (via CDP), and a real WeChat DevTools AppService, all running the *same* probe code so that observed differences are attributable to hosts rather than to probe drift; a snapshot-drift validator makes the pre-collected WeChat column auditable rather than merely trusted.

(4) **A two-level independent oracle** (§4.7): Level 1 compares entity observations across hosts; Level 2 executes the before/after programs in real hosts and checks observational equivalence—termination states, scalar results, and normalized exception identifiers—following the equivalence model of OBsmith [7] but across heterogeneous hosts.

Contributions. This paper makes the following contributions:

- **Problem and axiom.** We formalize the offline-folding hazard for heterogeneous deployments and state the required $\perp_{\text{target}}$ orthogonality axiom with a three-valued verdict design whose UNKNOWN deliberately dominates fold-under-doubt (§3–§4.4).
- **A complete, frozen v1 system** (§5): AST binding analysis with alias tracking, lexical-shadowing exclusion, and in-file-definition attribution; target-runtime resolution over a seven-state evidence lattice with seven hard confirmation gates; an eight-stage short-circuit decision funnel; and evidence assembly with validity windows—252 tests passing, architecture frozen against scope creep.
- **Empirical soundness at scale** (§6): across 11 benchmark configurations and three ecosystems, **zero** approved folds land on measured cross-host divergences (208/785 WeChat, 184/553 Node, 273/780 browser, plus a 154-entity hard set blocked with 100% recall); real Terser and webcrack rewrites that change behavior are all intercepted; controlled obfuscation of 112 real files (4,210 entity pairs) flips decisions only in the conservative direction, again with zero unsound folds.
- **Calibrated utility** (§6.2–§6.4): 75.2% approval under the record-weighted (natural-distribution) calibre and 30.6–43.3% under the entity-stratified calibre; the decidable rate rises from 33.9% to 48.3% once targets are confirmed—target confirmation converts "cannot decide" into "justified protection", not into more folding.
- **A methodology contribution** (§7): five extrapolation failures of cross-environment measurement, self-caught during development by five independent detection channels, with reusable checks ("existence may be extrapolated; absence must not be"; "the probe's world is not the analyzed program's world"; "one definition, one source"). We argue such a catalogue is a prerequisite for believing any soundness number in this problem space.

The rest of the paper is organized as follows. §2 develops the background and motivating scenarios. §3 fixes the threat model and problem statement. §4 presents the design. §5 describes the implementation. §6 reports the evaluation. §7 discusses methodology lessons and limitations. §8 surveys related work, and §9 concludes.

2 Background and Motivating Example

2.1 Why Environment Sniffing Is Ubiquitous

JavaScript's `typeof` operator returns `"undefined"` for undeclared identifiers instead of throwing a `ReferenceError`. This "free pass" is what makes `typeof X !== 'undefined'` the canonical cross-host dispatch construct: it lets one bundle probe its surroundings safely in every host. The idiom appears wherever a single code base must adapt to heterogeneous execution surfaces. We illustrate five recurring patterns drawn from production frameworks and prior systematizations of mini-program and evasion code [10, 11, 15].

(1) Transport-layer adapters (cross-platform network SDKs). The bundle selects its network primitive by sniffing host objects, exactly as in Listing 1: `wx.request` for mini-programs, `XMLHttpRequest`/`fetch` for browsers, `http` for Node. In an offline Node sandbox the first two branches are folded away; the production bundle loses its networking on the first two host types it was written for.

(2) Storage and monitoring fallbacks. Client-side SDKs (analytics, crash reporting, session persistence) select storage backends by host:

Listing 2: Storage fallback selection.

```
var storage;
if (typeof window !== 'undefined' && window.localStorage) {
    storage = window.localStorage; // browser main thread
} else if (typeof wx !== 'undefined' && wx.setStorageSync) {
    storage = wxStorageAdapter; // WeChat mini-program
} else {
    storage = memoryFallback; // Node SSR / sandbox
}
```

Folding `typeof window` to `"undefined"` in a bare sandbox silently downgrades every client to the in-memory fallback: the code *runs*, which makes the damage harder to notice than a crash—credentials and telemetry simply stop persisting.

(3) Worker-vs-main-thread dispatch. Web Workers and main threads have mutually exclusive globals (`importScripts`/`postMessage` vs. `document`/`window`). A bundle containing both render and compute paths must dispatch on presence; offline evaluation in a DOM-less sandbox prunes exactly one of the two paths, and which one is pruned depends on which sandbox the analyst happened to use.

(4) Evasion under analysis. Hostile code probes analysis artifacts—`navigator.webdriver`, `window.chrome`, `outerWidth > 0`—and releases its payload only when none are detected [11]. When an analyst deobfuscates such code in a minimal sandbox, every evasion probe evaluates to "absent"; a folding deobfuscator concludes the *payload branch is dead* and deletes it, leaving the harmless decoy. Here the mismatch does not merely break the redeployed application—it silently corrupts the security analysis itself.

(5) Multi-vendor mini-program sniffing. Multi-vendor frameworks emit parallel branches guarded by `wx` (WeChat), `my` (Alipay), and `tt` (ByteDance). All three objects are `undefined` in an offline Node sandbox, so a single offline evaluation deletes *all three* authentication paths at once.

In every scenario the deobfuscator's local evaluation is *locally correct*—the values it computes are the true values in its own sandbox—and *globally destructive*, because the values are not the target host's.

2.2 Why Hosts Genuinely Differ

It is tempting to believe that "pure ECMAScript behaves identically everywhere" and only obvious host objects (`wx`, `window`, `process`) diverge, so a small blocklist should suffice. Both beliefs fail against measured reality.

First, **hosts patch language-level objects**. WeChat's AppService adds `Math.nv_*` animation helpers that exist nowhere else, re-routes `console`, and wraps `Function` (reading `Function` on AppService stack-overflows the automation session—our probe excludes it explicitly and records the exclusion rather than pretending coverage). A folding decision about `Math.max`-adjacent code must therefore know which host it will run on *even for names that look like pure language built-ins*.

Second, **host objects hide from introspection**. On AppService, `wx.cloud` returns an empty result for both `in` and `Object.keys` while a direct property read returns a real function. An implementation that uses `in` to short-circuit "existence" reports host APIs as nonexistent, which then *matches* the language baseline's nonexistent—and matches its way into an unsafe fold. Our probe consequently uses `in` only as a confirmation for reads that returned `undefined`, never as a shortcut.

Third, **cross-host disagreement reaches into "standard" territory**: Node exposes `SharedArrayBuffer` while Edge hides it on non-cross-origin-isolated pages; legacy `RegExp.$1`-style accessors observe differently under descriptor versus direct-access probing. The probe matrix (§4.6) is designed to *measure* this surface, not assume it—and the measurement is why we can quantify it: 128 `wx.*` sub-paths, 15 `Math` members, 8 `process`/`Buffer` roots, and 7 language built-ins with host-specific behavior are among the 208 WeChat-relevant divergent entities (§6.1).

2.3 Why the Fix Cannot Be "Run Everything for Real"

The fully general check—execute each candidate folding point in the real target host and compare—is destructive and infeasible. *Destructive*: host APIs make network requests, pop payment dialogs, trigger permission prompts; some merely *crash the measurement session* (the `Function` stack overflow above). *Infeasible*: a mid-sized bundle offers tens of thousands of folding candidates; launching a browser or a DevTools session per candidate turns seconds of deobfuscation into hours, and the side effects of the probes themselves invalidate the analysis. Forced-execution engines [11] pay a related price—forcing an unsatisfied branch desynchronizes program state and can crash the engine.

R97 therefore splits the problem. A **cheap, safe, pre-computed observation layer** (Level 1) reads only metadata (existence/type/callability) for every entity the analysis touches, in every host; an **expensive, real-execution verification layer** (Level 2) runs only where it matters—on the folds R97 actually approves—and checks observational equivalence end to end. Between the two sits the three-valued verdict, which routes every folding point to exactly the evidence it needs, or to a justified UNKNOWN. What cannot be decided is *blocked with an attributed reason*, never guessed.

3 Threat Model and Problem Statement

Setting. An analyst runs a deobfuscator or minifier over a minified or obfuscated JavaScript bundle. The tool's constant folding and dead-code elimination evaluate expressions in the analyst's sandbox (Node.js by default). The bundle will be redeployed to one or more *target hosts*: known (packaging manifests present), declared (analyst assertion), or unknown (nothing external available).

Adversary and failure sources. The primary "adversary" is the environment mismatch itself—accidental (legitimate cross-host dispatch, §2) or crafted (evasion probing). Secondary adversaries are *stale* deployment evidence (the target host evolved after evidence was collected) and *forged or contradictory* evidence (mutually exclusive host declarations planted in the bundle). R97 is not designed to withstand an adversary that fully controls the analyst's machine; it assumes the deobfuscation tool itself is not malicious.

R97's contract. For each candidate folding point, R97 returns a verdict in {FOLD, PROTECT, UNKNOWN} with machine-readable reason codes:

- FOLD authorizes the fold *with respect to runtime semantics*: the evaluation environment's observation of every required entity is indistinguishable from every confirmed target host's observation, on every dimension the point's usage context demands.
- PROTECT blocks the fold because a *measured conflict* exists (host divergence, forbidden side effect, or evidence conflict).
- UNKNOWN blocks the fold because evidence is insufficient; the uncertainty source (`binding` / `runtime` / `behavior`) is attributed so the gap is auditable and, in principle, closeable.

PROTECT and UNKNOWN have identical enforcement (BLOCK_FOLD) but different knowledge content; the distinction is what makes "why wasn't this folded?" answerable and what separates "the system knows this is dangerous" from "the system lacks evidence".

Scope boundaries. R97 checks *runtime-semantics migration only* (`decisionScope = runtime_semantics_only`): language-level transformation correctness (scope hygiene, evaluation order, syntax) remains the downstream rewriter's responsibility, and R97's FOLD explicitly does not certify it. R97 does not execute the analyzed application; its probes read metadata only. R97 does not model what it cannot measure: the WeChat column is an offline snapshot bound to SDK 3.17.3 and an artifact hash, with drift *validation* rather than pretended liveness.

Problem statement. Given (i) a bundle's AST with binding resolution, (ii) a set of target hosts with externally-sourced confirmation states, and (iii) a probe matrix of host observations over the bundle's referenced runtime entities, decide for each candidate folding point whether the offline evaluation's result can be *safely migrated* to every confirmed target—where "safely" means the transformation's observable behavior in each target host is equivalent to the original program's, to the extent observable by non-destructive metadata plus guarded execution. The decision must be conservative: in the presence of insufficient evidence it must block, and its blocking must be *attributed* rather than silent.

4 Design

4.1 Overview

R97's pipeline is:

Listing 3: R97 pipeline.

```
source -> AST parse -> binding/scope/alias analysis
       -> runtime-entity identification
       -> semantic-contract projection (per folding point)
       -> target-runtime resolution (external evidence only)
       -> evidence assembly (4-host probe matrix + validity windows)
       -> eight-stage short-circuit decision
       -> FOLD | PROTECT | UNKNOWN -> ALLOW_FOLD | BLOCK_FOLD
```

Every verdict carries machine-readable reason codes (e.g., `BINDING_UNRESOLVED`, `TARGET_RUNTIME_INFERRED`, `REQUIRED_RUNTIME_MISSING`, `CONTRACT_MISMATCH`, `EVIDENCE_EXPIRED`) and an uncertainty source (`binding` / `runtime` / `behavior`), so "why not folded" is always auditable and improvable layer by layer. Two structural choices run through the whole design: *evidence is attributed and versioned*, never implicit; and *every relaxation of the pipeline had to survive measurement*—several plausible relaxations were implemented, measured to be unsafe, and rolled back (§7).

4.2 The Orthogonality Axiom: required ⊥ target

Required: what the code needs. Static analysis answers this. An identifier that resolves to no local binding is classified against a single-source registry (`global-roots.mjs`) that partitions roots into `LANGUAGE_BUILTIN_ROOTS` (ECMAScript built-ins, host-independent by specification) and `HOST_GLOBAL_ROOTS` (host- or module-system-injected names). The registry is shared verbatim by the analyzer and the language probe: when the two components maintained separate lists, 27 divergent entries caused pure built-ins (`Uint8Array`, `isNaN`, `TypeError`) to be misclassified as unresolved globals—a drift bug that motivated the single-source rule. Stable aliases are resolved transitively (`const w = wx; w.request()` inherits kind, path, and origin from `wx`, remaining `runtime_global`); reassignment degrades the alias to a local. Lexical shadowing (a parameter named `wx`) produces *no* host dependency—an independently important subtlety, since WeChat's packager wraps every module as `define("path", function(require, module, exports, window, document, ...))`, shadowing browser globals as closure parameters—a property we also verify directly across our 1,572-package corpus; treating those parameters as host references would misattribute the entire ecosystem. Additionally, R97 attributes *implicit globals written by the analyzed file itself* (`wh = $gwh();` with no declaration keyword): these receive an `inFileDefinition` marker (`top_level_assignment` / `function_body_assignment`), because a whole class of "WeChat-private" names turned out to be compiled packaging artifacts created by the analyzed code, not host capabilities (§7, case 4).

Required is three-valued: `definite` (unshadowed runtime globals, module imports, stable aliases), `possible` (dynamic property access, unresolvable scope relations—e.g., `obj[k]()`), and `not_required` (pure language computation, locals, shadowed host APIs, and host objects passed as parameters that are only accessed statically). Only `definite` can participate in hard-conflict decisions; `possible` and `ambiguous` blocks stop at UNKNOWN.

Table 1: The target-evidence lattice. Only `confirmed` authorizes folds; only `confirmed`+conflict or `conflict` trigger hard protection.

state	meaning	fold?	protect?
`confirmed`	all seven hard gates met (§4.2)	✓	✓
`corroborated`	consistent evidence, missing one independent source	×	×
`context_unknown`	family confirmed, surface/version missing	×	×
`declared`	caller assertion, uncorroborated	×	×
`inferred`	derived from code symbols only	×	×
`conflict`	credible evidence disagrees	×	✓
`unknown`	nothing external available	×	×

Target: where the code runs. Only evidence *external to the analyzed code* may answer this, over a deliberately paranoid lattice (Table 1). The seven hard gates for `confirmed`: evidence originates outside the analyzed code; its source is authoritative (official project config, deployment config, real execution traces, or runtime fingerprints); it binds to the current file/entry/artifact rather than the whole repository; it names platform family, execution surface, and version profile; at least one *independent* corroboration exists and no credible evidence conflicts; it binds to the current revision, artifact hash, or validity window; and multi-target builds are explicitly enumerated. The bite of the lattice is in its negative rules: `inferred`—the circular argument "code has `wx` ⇒ target is WeChat"—never reaches the engine as a target; a caller that supplies targets without declaring their source gets `declared`, not `confirmed`; and experiment harnesses must declare `experiment_config` explicitly, which

Table 2: Contract dimension projection by transformation kind.

transformation	base dimensions	conditional additions
CONST_EVAL	existence, type	+return_value (host entities, used results); +callback; +async
CALL_EVAL	existence, type, callability	same three conditionals
BRANCH_PRUNE	existence, type	same three conditionals
DEAD_CODE_DELETE	side_effect	same three conditionals

is recorded in every decision query so that any verdict's dependence on the harness is auditable. An early implementation bug demonstrated why the gates must be *structural*: when one version-profile signal was accidentally copied into three evidence records, the "≥2 independent evidences" gate was satisfied by three copies of one observation and 354 projects instantly became `confirmed`—the fix (only the version-bearing record carries the profile) plus a regression test guard against "counting one observation three times".

Host-attribution coverage is measured, not assumed. The table that decides "which host must be pulled into the comparison" was audited by probing all 356 roots across the four hosts: three missing mappings were found and fixed (`setImmediate`, `clearImmediate` → Node; `MutationObserver` → Edge), and 17 candidates were deliberately *not* mapped after measurement—`close`/`stop` are too generic (mapping them would inject spurious hosts), and the `wh`/`$gwx` family is not a host capability at all (§7, case 4).

4.3 Semantic Contracts: Projecting Only What a Fold Needs

A fold of `typeof wx !== 'undefined'` does not need to know what `wx.request` *returns*; it needs to know whether the *observation* of `wx` could differ between evaluator and target. R97 therefore builds, per folding point, a **semantic contract**: the minimal set of observation dimensions determined by the transformation kind and the usage context. The design vocabulary has nine dimensions (existence, type, callability, return_value, exception, async behavior, callback behavior, side effects, permission behavior); each contract requests 2–6 of them (Table 2).

Two rules make the contract machinery sound rather than merely structured:

- **Probes are non-destructive.** The shared oracle probe publishes only the triple `[existence, type, callability]`, read via property access without invocation. The red line is empirical: invoking host APIs makes network calls and pops dialogs, and *reading* `Function` on AppService stack-overflows the session. Dimensions that would require invocation are deliberately not produced by any probe.
- **Non-determinism is trapped, not enumerated.** If a CALL_EVAL point targets a normatively non-deterministic language built-in (`Date.now`, `Math.random`, no-arg `Date`), the contract demands `return_value`—a dimension no probe produces—so the decision lands in UNKNOWN *by construction*. This is a specification-level rule (ECMA-262 [2]), not a blocklist; it was not speculative design but the fix for two genuinely unsafe folds that Level-2 execution caught (`Date`, `Date.now` folded to fixed timestamps). Host-side analogues (`performance.now`, `crypto.randomUUID`) are *not* added to the trap: they are host capabilities, governed by the host-probing path—preserving the language-vs-host orthogonality even inside the trap rule.

Comparison uses `strict_equal` on projected values. The *asymmetric absence rule* governs the pathological case where the target host reports non-existence: because probes run in bare environments (not the analyzed application), an application-injected global (`wx.BaaS`, `window.SomePolyfill`) is legitimately absent from every probe world, and treating that absence as the target's truth collides with the evaluation baseline's absence into an unsafe "agreement" (§7, case 3 quantifies the damage: 372 records). R97 therefore discards target-host absences for host-attributed entities as `EVIDENCE_MISSING` (→ UNKNOWN), never as ground truth.

4.4 The Eight-Stage Short-Circuit Decision

The funnel evaluates, in fixed order, and short-circuits on the first failure:

1. **Binding gate**: unresolved or dynamic bindings → UNKNOWN.
2. **Contract gate**: missing or incomplete contract coverage → UNKNOWN.
3. **Required gate**: non-`definite` required → UNKNOWN.
4. **Target-confirmation gate**: target not `confirmed` while a `definite` required exists → UNKNOWN (the required runtimes *extend the candidate set* for future evidence; they never hard-protect on their own).
5. **Hard conflict**: `confirmed` target + `definite` required + measured conflict → PROTECT.
6. **Evidence gates**: missing, expired, conflicting, or under-sourced evidence → UNKNOWN.
7. **Contract comparison**: measured divergence or forbidden side effect → PROTECT.
8. **All-targets match** with fold-eligible evidence → FOLD.

The ordering encodes two audited invariants. First, stage 4 precedes stage 5: a weakly-sourced target must not hard-protect before confirmation gates run—this was itself a fixed defect (weak sources previously triggered hard blocks early, conflating "unverified" with "verified dangerous"). Second, the empty-target case cannot fold vacuously: an early version treated "no target to compare" as "all targets match"; the fix requires language-only releases to reference entities that *exist in the evaluation baseline*, which moved the target-unknown configuration's approvals from 1,419 to 1,262 and eliminated a class of vacuum folds that independent differential execution subsequently measured as 66 mis-approvals under a language-only configuration.

4.5 Resolving the Target from External Evidence

For mini-program packages, R97 derives family-level evidence from packaging artifacts themselves: co-presence of `app-service.js` (logic bundle) and `page-frame.html` (render frame) is a WeChat-packer fingerprint (grade `build_manifest`); compiled `app-config.json` with entry/page fields is an official-config signal; `project.private.config.json`'s `libVersion` yields a version profile when concrete (placeholder values like `trial` are rejected). Evidence records carry provenance grades, artifact hashes (SHA-256), scope bindings, and validity ranges; agreement checks compare a field only when *both* sources provide it—an early bug that compared missing against present would have made any multi-channel deployment permanently unconfirmable, since "missing ≠ conflicting" must hold for the lattice to be climbable.

On the 1,572 real packages this resolves 1,570 families (99.87%), 354 version profiles (22.52%), and—by design—**zero** `confirmed` targets, because no package carries a second independent source. We insist on reporting these as three numbers: 99.87% says "we recognize the package family", not "we can make final semantic decisions for these packages"; collapsing the three into one "resolution accuracy" would misrepresent the remaining gap as closed.

4.6 The Four-Host Probe Matrix

One implementation (`r97-oracle-probe`) runs in four hosts, so observed differences are attributable to hosts rather than probe drift:

- **Language baseline**—an isolated `vm` context serving as the deobfuscator's evaluation reference. Two subtleties: Node injects `console` into fresh contexts (removed explicitly), and `globalThis` must be redirected to the bare context—otherwise `globalThis.console` appears to be a *language* built-in, which produced 2 genuinely unsafe folds in the Node corpus before the fix.
- **Real Node.js**—with CommonJS module bindings (`require`, `module`, `exports`, `__dirname`, `__filename`) resolved per module semantics rather than `globalThis` reads. Reading them off `globalThis` made the verdicts depend on the *launch mode* of the probe (`node -e` vs. ESM script)—the same corpus flipped five PROTECT verdicts based on how the probe was started.
- **Real Edge** via the DevTools protocol, with per-run profile directories (a shared profile directory deadlocked consecutive runs with exit code 21—an operational bug that masqueraded as flaky tests).
- **Real WeChat DevTools AppService** (SDK 3.17.3), collected via the automation protocol with an *AppService-readiness gate*: on cold start the base library has not yet mounted `$gwx/wh`, and probing immediately would record "not yet loaded" as "nonexistent"—an unsafe-direction error. The collector polls for readiness signals and fails loudly rather than producing a premature report. The WeChat column is a *snapshot* bound to SDK version, collection time, and artifact hash; a drift validator re-collects the same entity set and classifies changes by safety direction (`declined` = dangerous, non-zero exit; `improved` = conservative; `neutral`), with 1,240/1,240 identical across two real collections.

The `existence` dimension itself had a caliber bug worth recording: equating "path resolved" with "value ≠ undefined" misreported the language built-in `undefined` as nonexistent, fabricating 114 over-protections before the fix.

Coverage is structural rather than curated: language/Node/Edge probes are collected *on demand* for whatever entities the current analysis touches; the pre-collected WeChat snapshot covers 1,238 of 1,248 corpus entities (99.2%), the 10 missing being the deliberate `Function`-family exclusions, each explicitly listed in the report.

4.7 The Two-Level Independent Oracle

Level 1 asks whether *entity observations* migrate across hosts. Level 2 asks the question that actually matters: *does the transformation preserve behavior in the target host?* It executes P (the original fragment) and $T(P)$ (the fragment with the evaluation-baseline result written in as a literal) in real hosts and compares observable behavior—normalized termination states, scalar results, and normalized exception identifiers—following the observational-equivalence model of OBsmith [7], across heterogeneous hosts.

Two implementation disciplines proved load-bearing. First, $T(P)$ **must encode what the evaluation baseline computes**, not the target's true value: folding *means* writing the evaluator's result into the code, so a "folded" form carrying the target's value fabricates a safe transformation and measures nothing (three designed cases made exactly this mistake and self-check assertions caught them). Second, **branch-pruning compares truthiness, not serializability**: `if (globalThis)` behaves identically before and after a truthiness-based fold even though `globalThis` serializes differently across hosts—an early implementation reported it as unsafe until the comparison was narrowed to what the transformation actually assumes.

The corpus-scale Level-2 run repaid the investment by catching a real R97 gap—non-deterministic calls (`Date`, `Date.now`) folding to fixed timestamps—which became the `return_value` trap of §4.3. Self-check assertions ("every case's folded form must reproduce the original's baseline result") remain in the test suite as a permanent guard against fabricated test cases, including the authors' own.

5 Implementation

R97 is implemented in Node.js (≥20) in roughly 30 core modules: an espree-based analyzer (AST, scope/binding resolution, alias-chain tracking, lexical-shadowing exclusion, implicit-global attribution, bounded constant propagation), the contract builder, the target-runtime resolver, the eight-stage decision engine, the evidence builder, four host probes plus the shared oracle probe, the WeChat DevTools collector (with readiness gating and drift validation), and the pipeline glue. The evaluation harness comprises 14 pipelines: main benchmarks with dual sampling calibres and project-cluster bootstrap confidence intervals, evidence-layer and strategy ablations, unknown-breakdown attribution, target-confirmation impact, differential reference execution, external-tool baselines (Terser/webcrack/ESLint), a semantic mutation benchmark, the two

transformation oracles, the host-attribution audit, the obfuscation-invariance benchmark, and a memory benchmark.

External corpora are fetched with pinned commits and per-file SHA-256 manifests; licenses are recorded verbatim (WPT: BSD-3-Clause; Node: MIT; application repositories: MIT/Apache-2.0/BSD-2), so every dataset byte is reproducible and auditable. The test suite—252 tests, run serially to avoid browser-probe timeouts—includes the OBsmith-derived regression suite covering 11 real-world obfuscator bug classes, the 21-case mutation benchmark asserting analyzer-level invariants under binding-shadowing, alias-resolution, dynamic-property, runtime-attribution, and transformation-aware mutations, and WeChat Worker sub-environment consistency checks.

6 Evaluation

We organize the evaluation around seven questions.

- **RQ1 (Soundness)**: do approved folds ever land on measured cross-host divergences?
- **RQ2 (Utility)**: how much legitimate folding does R97 preserve?
- **RQ3 (Design necessity)**: does each design element carry its weight?
- **RQ4 (Target confirmation)**: what does confirming the target actually buy?
- **RQ5 (Obfuscation invariance)**: do verdicts survive realistic obfuscation?
- **RQ6 (Real-world integration)**: does the guard intercept real tools' unsafe rewrites?
- **RQ7 (Cost)**: what are the throughput and memory profiles?

Datasets. Three ecosystems serve distinct roles. The WeChat corpus is the primary domain: 1,572 real packages (1,519 with usable JavaScript), 1,583 files (100% parsed), 1.2M program points, 1,248 distinct runtime entities—the only ecosystem where a full four-host closed loop with deployment truth exists. The Node corpus (nodejs/node test suites at a pinned commit, 900 files; plus express/webpack/eslint/fastify/node-fetch at pinned commits, 349 files) and the browser corpus (W3C web-platform-tests at a pinned commit, 900 files; plus jQuery/Chart.js/Leaflet/video.js, 377 files) provide the cross-ecosystem validation with the highest differential densities (553 and 780 measured-divergent records respectively).

Sampling calibres. The corpus distribution is extremely long-tailed: `Object.defineProperty` occurs 217,346 times while `setImmediate` occurs twice. Any record-uniform sample is dominated by a few high-frequency entities—and, critically for soundness, *rare dangerous entities fall out of it*: a 3,000-record uniform sample covers only 229 entities and contained **zero** differential entities, making every safety ablation vacuous. We therefore report two calibres that answer different questions and never merge them: **entity-stratified** sampling (per-entity reservoir cap 5 → 3,895 records covering 1,246/1,248 entities; rare entities like `setImmediate`, `Buffer.from`, `wx.request` guaranteed present) for soundness and coverage, and **record-weighted** sampling (3,000 records, uniform over the corpus) for natural-distribution utility. Two corpus entities (`default`, `default.hex_sha1`) are excluded from evaluation because `default` is a reserved word and cannot form an expression.

Table 3: Measured cross-host divergences and R97 soundness.

corpus	diff. entities/records	unsafe folds
WeChat mini-programs (3 hosts)	40 / 137	**0**
WeChat mini-programs (+real WeChat)	**208 / 785**	**0**
Node.js official tests (3 hosts)	184 / 553	**0**
Browser WPT (3 hosts)	**273 / 780**	**0**
Record-weighted sample	5 / 29	**0**

Statistics. Confidence intervals use project-cluster bootstrap (1,046 clusters, 2,000 resamples); treating records as independent would understate uncertainty, since one entity can contribute tens of thousands of occurrences.

Oracle scoping. A differential record counts as an *unsafe fold* only when all oracle hosts are inside the declared target set. This rule is not pedantry: in a `language+node`-only configuration, R97 approved 6 `SharedArrayBuffer` folds that Node-vs-Edge differential execution shows to be behavior-changing (Edge hides `SharedArrayBuffer` on non-cross-origin-isolated pages); adding Edge to the target set converts the same records to justified PROTECT. All soundness benchmarks therefore run with `-include-browser` and record an `oracleInScope` field.

6.1 RQ1: Soundness

Ground truth. The same oracle probe executes in all four hosts; an entity is *differential* if any two hosts disagree on any shared dimension. This includes the evaluation baseline: a difference between the deobfuscator's sandbox and any host is exactly the "cannot migrate" condition the system exists to catch.

Table 3 reports the headline result. Two observations sharpen it. First, the browser corpus is the stress test: 30% of its records are measured-divergent, and none of the 374 approvals lands on one. Second, adding WeChat to the *ground-truth panel* is not a formality: without it, 168 differential entities are invisible, of which 128 are `wx.*` sub-paths that Node and Edge cannot see at all, and 15 are the WeChat-patched `Math` members (`Math.nv_*`) and 7 rerouted/wrapped built-ins (`console`, `Function`, `Date`, `Uint8Array`, `eval`, `globalThis`, `RegExp`) whose divergence only the real host reveals. Among the 785 four-host differential records, 76 sit on folds that the three-host configuration *approved*—all 76 are exactly the records where not measuring WeChat means folding wrongly; R97's four-host verdict blocks them all.

Hard set: recall, not coverage. The main benchmarks sample by corpus distribution and answer "how much was approved and did any of it escape". The complementary question is "of every entity we *know* is host-sensitive, did anything escape?" The hard set is derived mechanically, not hand-picked: every stratified-corpus entity whose root belongs to the host-attribution tables, plus all WeChat-private-looking names. It contains 314 entities, of which 154 are measured-divergent (132 WeChat public APIs, 11 cross-host language/module objects, 2 base-library-injected `__wxConfig`, and 9 *pseudo-divergences*—the `wh`/`$gwx` family whose "divergence" is an artifact of probe-project compilation, retained deliberately because their UNKNOWN verdicts only make the recall measurement stricter).

Table 4: Utility by sampling calibre. The two calibres answer different questions and are never merged.

calibre	approval	decidable
record-weighted (natural distr.)	**75.23%**	76.20%
entity-stratified, 3 hosts	34.35%	37.56%
entity-stratified, 4 hosts	30.63%	48.47%
file-level analysis, 4 hosts	32.53%	**53.40%**

Table 5: Strategy comparison on the 4-host benchmark (785 measured-differential records).

strategy	approval	diff. records accepted
fold-everything (traditional)	100%	**785/785**
fixed 147-symbol blacklist	80.08%	115
prefix blacklist	84.06%	239
protect-everything	0%	0 (zero utility)
R97	30.63%	**0**

R97 blocks all 154 (recall 100%). A separately-reported 12-case human stress suite (`setImmediate`, `Buffer.from`, `process.version`, `SharedArrayBuffer`, `wx.request`, `window.document`, `document.cookie`, `navigator.userAgent`, `console.log`, `clearImmediate`) contributes 10 measured divergences, all blocked (recall 100%).

Level-2 transformation oracle. Of 20 designed cases spanning four groups (cross-host divergence, baseline-missing-but-target-present, pure language, shadowing/aliasing), the 10 behavior-changing transformations are all blocked; the 9 safe releases show zero behavioral mismatch in the target host; 1 case is over-protected (`JSON.parse('{"a":1}').a` blocked for missing `return_value` contract coverage—a reproducible conservatism case that maps the boundary of the current contract vocabulary). At corpus scale, of 1,338 approved folds, 386 are literalizable and are executed before/after in real Node and Edge: **zero behavioral mismatches**. (The 952 skipped cases have object/function-valued results that no real folder would fold; treating serialization differences as behavior would fabricate mismatches—the comparison therefore covers exactly what folding actually assumes.)

Mutation benchmark. Because corpus distributions under-sample analyzer edge cases, 21 semantically-mutated cases across five families (binding shadowing, alias resolution, dynamic property, runtime attribution, transformation-aware contracts) assert analyzer-level invariants with expectations derived from project invariants rather than from the implementation: 20/20 assertions pass, with 1 documented limitation (use-before-declaration aliases resolve conservatively—directionally safe).

6.2 RQ2: Utility

The record-weighted number (Table 4) answers "how much of real code's folding *volume* is preserved"; the stratified numbers answer "how much of the *entity space* is decidable". They must not be mixed, and neither is "the truth".

The low-but-decidable profile is not timidity, and the decidable rate—the share of records where the system has enough evidence to issue a justified verdict at all—is what separates two superficially similar profiles: Node-corpus decisions approve only 13.66% but *decide* 78.54% (mostly justified PROTECT; the system knows, and the answer is "no"); browser-corpus approval of 14.39% comes with only 30.28% decidability (WPT framework globals are unresolvable by design—100% of its UNKNOWNS are binding-layer). Adding real-WeChat evidence *raises decidability from 37.6% to 48.6% while slightly lowering approval*: evidence converts ignorance into justified protection, not into more folding.

At the entity level, of 1,246 entities, 260 (4-host) to 302 (3-host) are fully fold-safe; 38 (3-host) to 767 (4-host) carry justified protection; the remainder are attributed UNKNOWNS whose decomposition we examine in §6.4. Whole-file analysis (which resolves more context than the per-point synthesis used in the stratified benchmark) lifts approval to 32.53% and decidability to 53.40%.

6.3 RQ3: Design Necessity (Ablations)

Table 5: R97 strictly dominates protect-everything—identical zero unsoundness with +30.6pp utility. The blacklists are *not* dominated and we do not claim they are: they trade escaped divergences for approval, i.e., they sit on a different point of the safety–utility frontier, one that accepts 115–239 behavioral breaks to fold more. Which point is right depends on whether the redeployed bundle is allowed to break; R97's design goal—deobfuscation whose output must run—takes the zero-escape point.

The third value is load-bearing. Treating UNKNOWN as FOLD (a two-valued ablation) produces **32 unsound folds** on the same data where R97 produces zero. Only the entity-stratified calibre reveals this: the record-weighted sample contained zero differential entities, so the same ablation reported zero-vs-zero and the third value looked gratuitous. The sampling calibre is thus itself a validity condition for the ablation.

Evidence layers. A language-baseline-only configuration (the deobfuscator's implicit worldview) mis-approves **66** records—`console`, `setImmediate`, `Buffer` are invisible to a bare VM, so their cross-host differences vanish; adding Node as a target zeroes it, confirming that required-driven candidate extension forces every necessary host into the comparison rather than trusting the analyst to declare it.

Attribution coverage. The required-runtime table was audited by *measurement* (§4.2): 356 roots probed across four hosts found 3 missing mappings (fixed: `setImmediate`, `clearImmediate` → Node; `MutationObserver` → Edge) and 17 candidates deliberately rejected after measurement. A companion ablation of name-pattern-based "framework filtering" showed the tempting alternative is unsafe: a broad framework-name filter deletes 19.3% of call sites and 8 measured divergences, and even the narrowest useful variant removes nothing that matters—so the mechanism was excluded from the safety chain entirely.

6.4 RQ4: Target Confirmation

Two findings deserve emphasis (Table 6). First, **the platform resolver alone lowers approval** (1,419 → 1,199): pulling WeChat

Table 6: Effect of target confirmation, everything else fixed.

target state	FOLD	PROT.	UNKN.	decidable
unknown	1,419	0	2,476	32.40%
family confirmed	1,199	129	2,573	33.94%
fully confirmed	1,193	687	2,015	**48.27%**

Table 7: Obfuscation invariance under two controlled obfuscation configurations.

metric	value
shared entity pairs	4,210
decision consistency	85.84%
unsound folds (approved ∩ differential)	**0**
95% CI upper bound (unsound rate)	≈ 0.071%
dangerous transitions (PROT→FOLD, FOLD→PROT)	**0**

into the target set forces `Math.nv_*`-style tainted built-ins to be actually compared, closing 3 vacuous folds that existed only because "no target to compare" was treated as "all targets match". Correctness and coverage move in opposite directions here, and conflating them would produce a wrong conclusion ("target resolution hurts"). Second, **confirmation's value is classification, not approval**: comparing per-record against the `context_unknown` row, 557 UNKNOWN records (21.7% of 2,573) become justified PROTECT and **zero** become FOLD. Confirmation converts "cannot decide" into "knows it cannot fold"—which is precisely the conversion a guard should optimize.

The residual UNKNOWNs under full confirmation decompose into attributed, actionable buckets: 970 module-boundary (cross-module resolution frozen out of v1), 571 code-defined globals (implicit globals the analyzed file assigns itself—frozen local-function evaluation), 331 other unbound globals (must remain UNKNOWN for safety), 72 WeChat base-library-private globals, and 68 WeChat public API paths withheld by the absence-extrapolation rule (§4.3). The first two buckets—76% of all UNKNOWNs—are the frozen v1 analysis-unit boundary, a deliberate cost booked to coverage, never to safety.

6.5 RQ5: Obfuscation Invariance

The primary corpus is production-minified but not adversarially obfuscated, and a reviewer may reasonably ask whether verdicts survive real obfuscation. We therefore test **invariance**: 112 real files across the three ecosystems, size-stratified (small 3–10KB / medium 10–60KB / large 60–120KB), obfuscated by `javascript-obfuscator` under two configurations (M1: string-array 0.8 with rotation/shuffling; M2: M1 + control-flow-flattening 0.7 + dead-code-injection 0.3), with `renameGlobals:false` so entities pair across versions (obfuscation that renames globals would make cross-version entity matching meaningless; it is also non-default in the tool). Each original/obfuscated pair runs the full pipeline with four-host evidence.

All 596 inconsistencies (Table 7) migrate conservatively, and the *mechanism* is legible: FOLD→UNKNOWN 538 (obfuscation wraps references in alias chains and string-array decoder calls, breaking binding resolution—`BINDING_UNRESOLVED` rises by 533–1,210 depending on configuration), PROTECT→UNKNOWN 20 (hard blocks degrade to soft; both still block, so enforcement is unchanged), UNKNOWN→FOLD 38 (usage-context changes after rewriting; every one verified off the differential set). Obfuscator-introduced helpers—the string-array decoders and flattened-dispatch machinery—land overwhelmingly in UNKNOWN (167/180 in the heaviest configuration): the guard refuses to fold the obfuscator's own machinery. The summary sentence for this experiment: **obfuscation bends R97 toward caution; it never bends it toward risk.**

6.6 RQ6: Real Tools under the Guard

Using pinned real tools (Terser 5.51.2, webcrack 2.16.0) on controlled cases with three-host differential execution of the *actually rewritten* code: Terser produced 5 rewrites, all 5 behavior-changing—**all 5 intercepted** by the gate; webcrack produced 7 rewrites, 1 behavior-changing—**intercepted**. Static ESLint-based baselines (three global-table configurations, from language-only to all-hosts-declared) accept 22–100% of measured-unsafe folds on the same cases.

We deliberately do not report these as generic tool criticism: Terser and webcrack are not safety arbiters, and their folding is *correct* for the homogeneous deployments they were designed for. The meaningful metrics are exactly two—"behavior mismatches after real rewriting" and "mismatches still accepted after adding the R97 gate"—and on both, the gate reduces the second number to zero without forbidding the tools from folding what is safe.

6.7 RQ7: Cost

Decision throughput is 926–3,147 records/s depending on configuration (the micro-decision including evidence matching is ≈0.12 ms); whole-file analysis sustains 10–12 large bundles/s; the four-host probe matrix amortizes to a per-benchmark cost (language/Node probes are in-process; Edge is one CDP session; WeChat is a pre-collected snapshot). Under `–expose-gc`, whole-file analysis of 100 real `app-service.js` bundles shows a **+8.2 MB steady-state heap** with no accumulation across files (+3.9 MB for streaming 3,900-record corpus reads; peaks include non-returnable V8 pages and are reported separately in the artifact). Repeatability: Node probe—20 repetitions, one unique snapshot; Edge—5 repetitions, one unique snapshot; WeChat—two independent collections, 1,240/1,240 entities identical (0.00% drift).

7 Discussion

7.1 Five Extrapolation Failures—and Why We Publish Them

Our soundness numbers invite a specific distrust: the scoring probe shares implementation with the decision chain, so "0 unsound folds" could be self-consistency theater. We treat the evaluation infrastructure itself as a study object. During development, five failures were caught—each by an *independent detection channel*, none by manual review—and each is one facet of a single root cause: **treating a fact observed in one environment as the truth of another**.

(1) Evaluation-reference pollution. The language baseline originally ran in a Node realm, so `globalThis.console` looked like

a language built-in; the Node corpus registered 2 genuinely unsafe folds before the mismatch was traced. *Detection*: consolidating the baseline into a single module; *fix*: isolated VM + explicit `globalThis` redirection.

(2) Probe-environment misattribution. A fixture mini-program's own compiled globals were taken for WeChat host capabilities; attributing them into the host list inflated FOLD by 529 records—a *directionally unsafe* change that looked conservative because it only added a host to the target set. *Detection*: namespace auditing; *disposition*: implemented, measured, and **rolled back**, with a standing rule: never widen host lists to raise approval.

(3) Absence extrapolation. Probes run in bare environments, so application-injected globals (`wx.BaaS`, `window.SomePolyfill`, `navigator.X`) are absent from every probe world; treating that absence as the target's truth collides with the evaluation baseline's absence into unsafe "agreement". The measured damage: **372 potential unsafe approvals** (68 WeChat—including `wx.loadSubpackage`, `wx.createCanvas`; 110 browser—`window.*`/`navigator.*`; 194 Node—`process.env.*`). *Fix*: the asymmetric rule—only host-attributed entities whose target observations are *all* absent have those observations discarded (`EVIDENCE_MISSING`), never used as ground truth; evaluation-baseline absences remain fully usable, since the evaluator's world is exactly what folding writes in.

(4) Name misattribution. The `wh`/`$gwx`-family names—probed as "existing and callable in WeChat"—are in fact implicit globals *assigned by the analyzed files themselves* (verified 1,496/1,496 files for `wh`; the probe saw them because the probe project is itself a compiled mini-program with same-named artifacts). 571 records were re-attributed to `inFileDefinition`, and the tempting remedy "run the probe inside the analyzed project" was **measured and rejected**: inside the project, the probe would see the very object the code just created—not a host capability. Note the meta-lesson: failure (2)'s remedy was *negated* by failure (4)'s measurement.

(5) Definition drift. The language baseline existed in two implementations—one patched to remove Node-injected `console`, one not—so the independent differential executor's language column silently diverged. *Detection*: a self-check assertion in the Level-2 case set; *fix*: one definition, one module.

The catalogue yields a reusable claim: **a cross-environment soundness number is meaningless until the extrapolation authority of every observation has been audited**. Existence may be extrapolated across hosts; absence must not. The probe's world is not the analyzed program's world. A definition written twice will drift. Each check is tool-independent; all five are encoded as standing tests.

7.2 Honest Scope of the Safety Claim

"0 unsound folds" is stated under two complementary calibres, and neither is a formal proof. The **closed-loop calibre** (decision rules vs. their own evidence rules) proves no rule violation but shares implementation with the ground truth. The **independent calibre** (four-host measured differentials + Level-2 execution) does not share the decision logic but measures only what the probes and execution cover. Three boundaries must accompany any citation of the number: (i) WeChat is an offline snapshot bound to SDK 3.17.3—drift-validated (0.00% across two real collections) and version-keyed, but not live; (ii) the `Function` family is structurally unmeasurable on AppService (session-killing stack overflow) and is excluded by name, not silently uncovered; (iii) a 500-record human-annotation subset (two-annotator, Cohen's κ pipeline) is scaffolded and pending, after which the exact-state accuracy claim gains a fully independent anchor. Over-protection is independently evidenced by exactly one case (`JSON.parse(...).a`); its broader rate is self-consistent by construction and labeled as such everywhere it appears.

7.3 Known Limitations

- **Analysis-unit boundary (frozen)**: v1 performs no cross-module resolution and no local-function evaluation. Together these account for 1,541 of 3,895 stratified records (39.6%) landing in UNKNOWN—1,541 records that are *blocked with an attributed reason*, a cost booked entirely to coverage, never to safety.
- **Single-file evaluation in the stratified benchmark**: the point-synthesis used for sampling discards original scope; the file-level configuration (§6.2) recovers part of the gap and is the recommended deployment mode.
- **Worker sub-environments**: six real Worker reports (4 independent projects + fixture + one repeat) show a single consistent divergence signature (`wx`, `wx.request`, `Page`, `getApp`, `document` absent in Workers), but the 12-entity table is *not* fed into decisions—refusing to extrapolate from a 12-entity probe is the same absence-extrapolation discipline as §7 (3).
- **0% confirmed targets in the wild**: single-source version profiles cap at `corroborated`; the deployment-fingerprint channel for a second independent source exists (collector, evidence builder, validity windows) but requires in-field collection across deployments.
- **Guard granularity**: R97 blocks at folding-point granularity, not file granularity—a PROTECT verdict never halts deobfuscation of the whole file, which preserves utility but means the *analysis* of a protected branch continues (harmless: the branch is preserved verbatim).

8 Related Work

Deobfuscation and simplification. JStill [14] pioneered mostly-static analysis of obfuscated malicious JavaScript, showing how much obfuscation structure survives static scrutiny—the insight that grounds the static half of our pipeline; Lu and Debray [9] established semantic preservation as the field's first principle—"rewriting must not change the program's semantics". JSIMPLIFIER [16] and webcrack [4] industrialize deobfuscation for security analysis, and Terser [1] performs the same folding in production minification. All evaluate in a single local sandbox; R97 is the guard that makes their offline world-check explicit, and its verdicts are designed to compose with their pipelines rather than replace them.

Obfuscator testing. OBsmith [7] tests *obfuscators* (the opposite direction of the pipeline) with LLM-generated program sketches and

observational equivalence, uncovering 11 industrial bug classes. Its equivalence formalism—termination state, output projection, exception identity—directly informs R97's Level-2 oracle; its homogeneous-single-runtime assumption (all comparison inside one local Node process) is exactly what R97 removes, since a bundle referencing `wx` cannot even execute its original form in OBsmith's world. Its negative result on metamorphic testing (zero independent yield beyond reference equivalence) informed our decision *not* to import metamorphic relations.

Dynamic analysis of hostile JavaScript. FV8 [11] forces evasive branches to execute inside a modified V8, at the cost of state desynchronization the authors themselves flag; R97 never forces execution—it blocks and preserves, trading coverage for stability. VisibleV8 [8] instruments V8 to observe real-browser API behavior, providing independent evidence that host surfaces diverge in practice. Wemint [10] analyzes WeChat mini-program architecture—the dual-thread model, packaged module structure, and the `wx`/`App`/`Page` root system—and its leak-tainting analyses build on exactly the packaging conventions that motivate our lexical-shadowing exclusions. AppSecret-leak studies [15] document the mini-program threat surface from the credential side, complementary to our deployment-semantics focus.

Hybrid static/dynamic analysis. DONAPI [3] (malicious npm packages, hybrid behavior sequences) and JScamd [5] (cryptographic API misuse via static taint analysis) demonstrate that neither pure-static nor pure-dynamic analysis suffices on obfuscated JavaScript, and that fine-grained API context beats global name matching. R97 borrows the hybrid architecture and the contract *idea*, but its contracts are over *folding points* with host-differential semantics—dimensions projected per transformation and usage—which has no analogue in misuse detection.

Translation validation. R97 instantiates the gatekeeper pattern of translation validation [13] at transformation granularity: per-fold verification against *measured* host semantics rather than whole-compiler proofs. Where classical translation validation relates two programs' semantics via a proof, R97 relates two *environments'* observations via measurement—and its three-valued verdict exists precisely because measurement, unlike proof, can be incomplete.

Differential testing of engines. JEST [12] distributes one input to N engines plus a specification and converges on a central comparator; R97's four-host matrix with a shared probe is the same distribution-comparison topology applied to deployment hosts rather than engine implementations.

9 Conclusion

Offline constant folding silently assumes that the evaluator's world is the deployment world. For heterogeneous JavaScript—from WeChat mini-programs to WPT browser suites—that assumption fails measurably, 785 and 780 differential records at a time, and the failure mode is silent: the folded bundle looks clean and runs broken. R97 turns the assumption into an explicit, evidence-gated decision. Required⊥target orthogonality forbids the circular argument; minimal semantic contracts over non-destructive probes decide what must be compared; a seven-state external-evidence lattice with seven hard gates decides who may compare; an eight-stage conservative funnel decides with attributed uncertainty; and a two-level oracle—cross-host observation plus real-execution observational equivalence—checks the result against worlds R97 does not control. The result is a guard that approves 30.6–75.2% of folds by calibre with zero measured escapes at a 0.07% confidence bound, blocks all behavior-changing rewrites of real Terser and webcrack runs, survives controlled obfuscation with only conservative decision drift, and costs milliseconds per decision with megabytes of memory. Just as importantly, it documents—rather than hides—the five extrapolation pitfalls that any cross-environment measurement must audit before its soundness numbers can be believed: existence may be extrapolated, absence must not; the probe's world is not the analyzed program's world; and one definition, written twice, will drift.

Artifacts. Corpus builders (pinned commits, SHA-256 manifests, recorded licenses), the four-host probe matrix with snapshot-drift validation, all 14 evaluation pipelines, the obfuscation-invariance benchmark, and the 252-test suite are available for reproduction.

References

[1] Mihai Bazon et al. 2026. Terser: JavaScript parser, mangler and compressor toolkit for ES6+. https://github.com/terser/terser. Version 5.51.2, accessed 2026-10-12.

[2] ECMA International. 2024. ECMAScript Language Specification (ECMA-262). https://tc39.es/ecma262/. Accessed 2026-10-12.

[3] Cheng Huang, Nannan Wang, Ziyan Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. 2024. DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping. In *Proceedings of the 33rd USENIX Security Symposium*.

[4] j4k0xb. 2026. webcrack: Deobfuscate, unminify and unpack bundled JavaScript. https://github.com/j4k0xb/webcrack. Version 2.16.0, accessed 2026-10-12.

[5] Shijie Jia, Bowen Xu, Yuan Ma, Yingjiao Niu, Limin Liu, Daren Zha, and Jingqiang Lin. 2026. JScamd: An Automated Static Taint Analysis Framework for Detecting Cryptographic API Misuses in JavaScript. In *Proceedings of the 35th USENIX Security Symposium*.

[6] Shan Jiang, Pranoy Kovuri, David Tao, and Zhixun Tan. 2026. CASCADE: LLM-powered JavaScript Deobfuscator at Google. In *Companion Proceedings of the 48th International Conference on Software Engineering (ICSE-SEIP '26)*. 292–302. doi:10.1145/3786583.3786873

[7] Shan Jiang, Chenguang Zhu, and Sarfraz Khurshid. 2026. OBsmith: LLM-Powered JavaScript Obfuscator Testing. *Proceedings of the ACM on Programming Languages* 10, OOPSLA1 (2026), 114–142. doi:10.1145/3798204

[8] Jordan Jueckstock and Alexandros Kapravelos. 2019. VisibleV8: In-browser Monitoring of JavaScript in the Wild. In *Proceedings of the ACM Internet Measurement Conference (IMC)*. 393–405. doi:10.1145/3355369.3355599

[9] Gen Lu and Saumya Debray. 2012. Automatic Simplification of Obfuscated JavaScript Code: A Semantics-Based Approach. In *Proceedings of the 2012 IEEE Sixth International Conference on Software Security and Reliability (SERE)*. 31–40. doi:10.1109/SERE.2012.13

[10] Shi Meng, Liu Wang, Shenao Wang, Kailong Wang, Xusheng Xiao, Guangdong Bai, and Haoyu Wang. 2023. Wemint: Tainting Sensitive Data Leaks in WeChat Mini-Programs. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1403–1415. doi:10.1109/ASE56229.2023.00151

[11] Nikolaos Pantelaios and Alexandros Kapravelos. 2024. FV8: A Forced Execution JavaScript Engine for Detecting Evasive Techniques. In *Proceedings of the 33rd USENIX Security Symposium*. arXiv:2405.13175.

[12] Jihyeok Park, Seungmin An, Dongjun Youn, Gyeongwon Kim, and Sukyoung Ryu. 2021. JEST: N+1-Version Differential Testing of Both JavaScript Engines and Specification. In *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*. 13–24. doi:10.1109/ICSE43902.2021.00015

[13] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *Proceedings of the 4th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS), LNCS 1384*. 151–166. doi:10.1007/BFb0054170

[14] Wei Xu, Fangfang Zhang, and Sencun Zhu. 2013. JStill: Mostly Static Detection of Obfuscated Malicious JavaScript Code. In *Proceedings of the Third ACM Conference on Data and Application Security and Privacy (CODASPY)*. 117–128. doi:10.1145/2435349.2435364

[15] Yue Zhang, Yuqing Yang, and Zhiqiang Lin. 2023. Don't Leak Your Keys: Understanding, Measuring, and Exploiting the AppSecret Leaks in Mini-Programs. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2411–2425. doi:10.1145/3576915.3616591 arXiv:2306.08151.

[16] Dongchao Zhou, Lingyun Ying, Huajun Chai, and Dongbin Wang. 2026. From Obfuscated to Obvious: A Comprehensive JavaScript Deobfuscation Tool for Security Analysis. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2026.242198 arXiv:2512.14070.